

IntSeqBERT: Learning Arithmetic Structure in OEIS via Modulo-Spectrum Embeddings

Kazuhisa Nakasho

CICM 2026

Iwate Prefectural University, Japan

Code & data: github.com/aabaa/IntSeqBERT

Why Finding Good Problems Matters

Terence Tao on mathematical problems

I wrote recently about how the collection of **good, fruitful open problems** is now being mined in a non-renewable fashion, leading to the potential scenario of **these problems becoming scarce**. This may seem unintuitive at first, since the set of possible problems one could ask is infinite. Perhaps the following analogy can help: a country or region can suffer a **critical shortage of drinking water** while simultaneously being **surrounded by a massive ocean**.

One can easily generate any number of open problems in mathematics at will, such as working out the $10^{10^{10}}$ th digit of pi. But the vast majority of such problems are not worth focusing attention on: they show no particular propensity to **reveal any further insights or connections to other questions**, or may either be too easy or too impossible relative to known techniques to learn anything from the exercise. (1/4)

Source: Terence Tao, Mathstodon, post 1/4 (quoted in full; emphasis added).

mathstodon.xyz/@tao/117237320796901560

Long-Term Goal — AI for Mathematical Discovery

- **New mathematical connections often begin with numerical patterns.**
 - McKay noticed $196,884 = 196,883 + 1$; this led to *Monstrous Moonshine*.
 - The BSD conjecture and mirror symmetry also began with numerical patterns (317,206,375 rational curves in the mirror symmetry example).
- AI now takes part in this kind of discovery. Examples include **AlphaProof**, the **Ramanujan Machine**, and **FunSearch**.
- **Our long-term goal is to find similar arithmetic patterns across sequences and use them to suggest new conjectures.** The sequences may come from different fields of mathematics.
- We use the **OEIS**: 391,710 sequences, each with a precise definition. These definitions make mathematics accessible to computers.
- **As a first step, we build a representation that captures arithmetic structure.** We then measure how much structure it learns.

Why Multiplication Is Hard for Deep Learning

However, standard neural models struggle to learn multiplication rules.

With **spectral bias**, they learn low-frequency, addition-like patterns first (Rahaman+ 2019).

Model and task	What the result shows
Threshold circuits: n -bit multiplication (Siu & Roychowdhury 1994)	3 layers are sufficient and minimal. A small circuit exists; training need not find it.
GPT-4: decimal multiplication (Dziri+ 2023)	Accuracy falls from 59% for 3×3 -digit products to 4% for 4×4 -digit products.
2-layer Transformer: multiplication mod p (Power+ 2022)	It generalises only after a very long training period (grokking).
12-layer model: implicit chain-of-thought (Yao+ 2025)	It succeeds on 15-digit products with a special training method.

These models and tasks differ, so their layer counts are not directly comparable.

- **Tokenisation also hides arithmetic structure.** Token IDs do not show it directly; values outside the vocabulary become UNK.
- This matters for the OEIS: values range from single digits to 10^{210} .

Key Idea — Show Multiplicative Structure in the Input

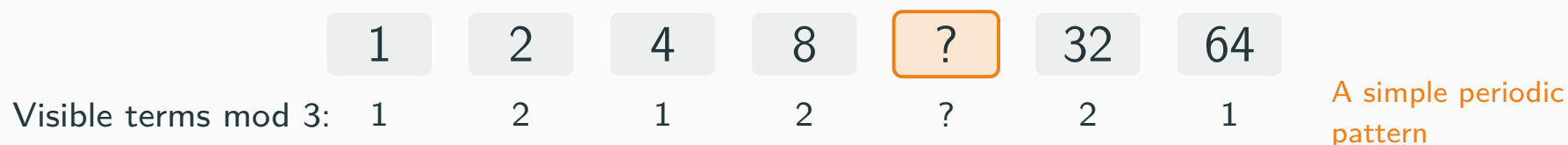
- **So we give the model residues, which preserve addition and multiplication.** A residue is the remainder after division. Reduction mod m is a **ring homomorphism**: $\mathbb{Z} \rightarrow \mathbb{Z}/m\mathbb{Z}$.
- **The residues stay small, even when the original values are very large.** For 10^{210} , a residue is still in $\{0, \dots, m-1\}$. The sequence $(x_i \bmod m)$ keeps multiplicative structure.
- Classical number theory explains these **periodic patterns**:
 - **Fermat's little theorem** describes powers mod p (the period divides $p-1$).
 - **Quadratic reciprocity** links residue information across primes.
 - The **CRT** combines information from several prime powers.
- **Our hypothesis is that these inputs make multiplication easier to learn.** We call them the *modulo spectrum*. They show multiplicative structure **at depth zero**, before the network learns it across layers.

Task — Masked Sequence Modelling on OEIS

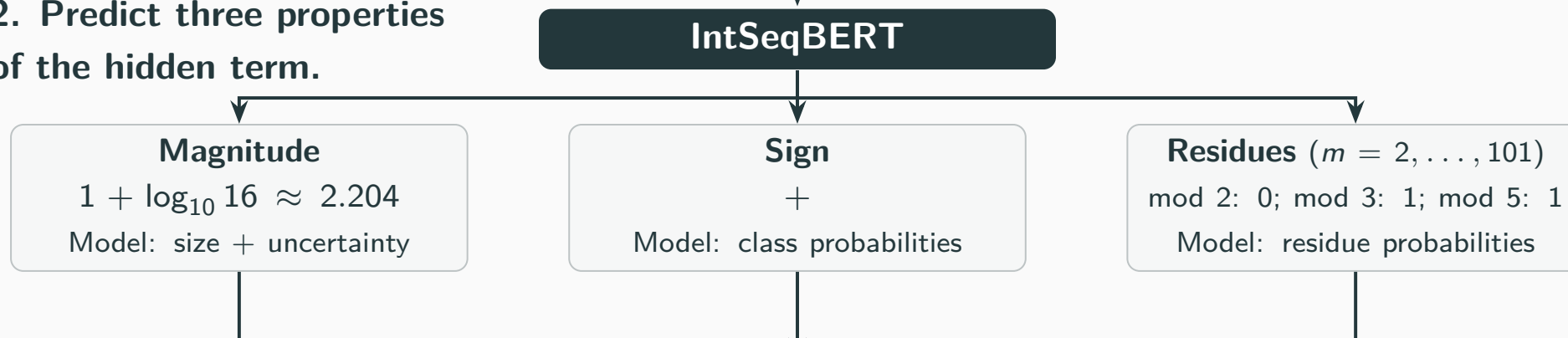
- To test this idea, we hide sequence terms and ask the model to predict them. This is **masked sequence modelling**: BERT-style learning for integers.
- We take up to the first **128 terms** and mask **15%** of the positions. The remaining terms provide the context.
- This tests how well the model captures patterns in the sequence. In **next-term prediction**, only the last term is masked.
- The **FACT** benchmark (NeurIPS 2022) tested a plain Transformer on five OEIS tasks. Unmasking was the hardest task. Our **Vanilla baseline** uses that architecture.
- For each hidden term, we predict its magnitude, sign, and residues.
 1. **Magnitude** measures log-scale size: $v_i = 1 + \log_{10} |x_i|$.
 2. **Sign** has three classes: $s_i \in \{+, -, 0\}$.
 3. **Residues** are $x_i \bmod m$ for every $m \in \{2, \dots, 101\}$.

A Worked Example — Predicting a Missing Integer

1. Hide a term; use the surrounding terms as context.



2. Predict three properties of the hidden term.



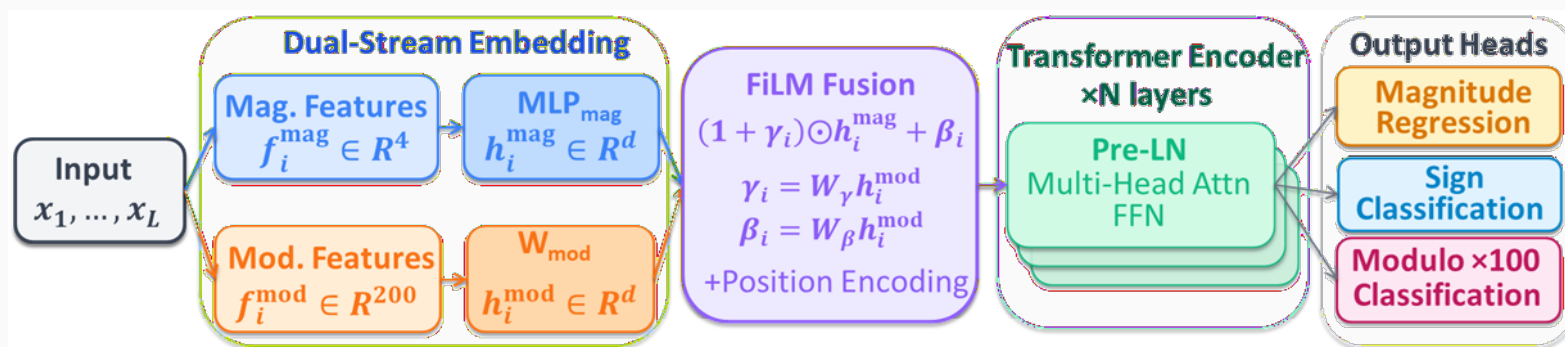
3. Combine predictions to recover an integer.



Values shown are training targets for 16, not measured model outputs.

Training: compare the three predictions with the hidden term's targets → update IntSeqBERT.

IntSeqBERT — Dual-Stream Architecture



- **The model uses two input streams: magnitude and modulo.** The magnitude stream sends log-scale size and sign ($\mathbb{R}^4$) through an MLP.
- The modulo stream ($\mathbb{R}^{200}$) places residues on the **unit circle**: $\phi_m(r) = [\sin(2\pi r/m), \cos(2\pi r/m)]$. This uses sinusoidal positional encoding for values' residues; residues $m-1$ and 0 stay neighbours.
- **We combine the streams and pass the result to a Transformer.** With **FiLM**, the modulo stream scales and shifts the magnitude stream: $e_i = (1 + \gamma_i) \odot h_i^{\text{mag}} + \beta_i$.
- We use a standard **pre-LN Transformer encoder**. Model sizes are Small 6.4M, Middle 29.0M, and **Large 91.5M** parameters.

Training — Multi-Task Objective

- We train the model to predict all three quantities together.
 - The **magnitude head** predicts size and its **uncertainty**, (μ_i, σ_i^2) . The Solver uses both.
 - The **sign head** predicts one of three classes.
 - The **modulo head** has 100 independent classifiers. Each loss is normalised by the maximum entropy $\ln m$, so mod 2 and mod 101 contribute fairly.
- We combine the losses with these weights: $\mathcal{L} = \mathcal{L}_{\text{mag}} + \mathcal{L}_{\text{sign}} + 2 \mathcal{L}_{\text{mod}}$.
- We train on **219,765 OEIS sequences** for **200 epochs**. We use **FP32**, because values near 10^{210} overflow FP16.
- **All experiments run on a single consumer GPU**. We use an RTX 3070 Ti with 8 GB, so other researchers can repeat the experiments.

Neural-Symbolic Decoding — the CRT Solver

- **First, the Solver generates candidates in the predicted magnitude range.** It searches the 3σ interval, using its size to choose a method:
 - **Dense** ($\Delta n \leq 10^6$) checks every integer in the interval.
 - **Sieve** ($\leq 10^{14}$) uses CRT beam search with confident residues.
 - **CRT** ($> 10^{14}$) generates large candidates directly with sparse CRT.
- **Next, it scores candidates using magnitude and all 100 modulo distributions.** For each m , it uses $P_m(n \bmod m)$.

Probabilistic CRT: We add weighted log-probabilities.

$$S(n) = \underbrace{-\frac{(v_n - \mu)^2}{2\sigma^2}}_{\text{magnitude log-likelihood}} + 0.3 \underbrace{\sum_{m=2}^{101} \ln P_m(n \bmod m)}_{\text{all modulo log-probabilities}}, \quad v_n = 1 + \log_{10} n$$

$P_m(r)$ is a residue probability. The weight 0.3 limits the effect of shared information.

- **Finally, it returns the candidates with the highest scores.** It takes the top k positive candidates and applies the sign; a zero sign returns 0.

Dataset — 274,705 OEIS Sequences

- We keep about 70 percent of the OEIS entries for this task. From 391,710 entries, we retain 274,705 (70.1%) and split them 8:1:1.
- We exclude eleven tags, grouped by the reason for exclusion.
 - Five tags depend on **how values are written**: cons, cofr, frac, base, word. Their patterns depend on the encoding.
 - Two tags **do not fit this task**: fini, tabl.
 - Four are **quality labels**: dead, unkn, less, dumb.
- These choices define our task. The labels less/dumb are subjective, and tabl could be converted into sequences. Future benchmarks may choose differently.
- We also group values by magnitude to compare easier and harder cases. The five **value buckets** are Small ($< 10^2$, 52.7%), Medium, Large, Huge, and Astronomical ($\geq 10^{50}$).

Metrics — and Three Questions

We measure size, residues, and exact integer prediction.

Metric	Definition	What it measures
Magnitude Accuracy ($\text{Acc}_{0.5}$)	Fraction within 0.5 of the true log-scale magnitude	Approximate size; helps set the Solver's search range
MMA (mean modulo accuracy)	Mean classification accuracy over all 100 moduli	Accuracy of residue predictions
NIG (normalised information gain)	$1 - \mathcal{L}_{\text{CE}}^{(m)} / \ln m$; 0 = no gain over uniform prediction; 1 = perfect prediction	Periodic structure learned for each modulus
Solver Top-k	Fraction with the true integer among the top k candidates	Exact integer prediction

For magnitude accuracy, an error within a factor of $\sqrt{10} \approx 3.16$ is allowed.

True value $10^6 \rightarrow$ predictions in about $[3.2 \times 10^5, 3.2 \times 10^6]$ count as correct.

We ask three questions: does the dual stream help, where does it help, and can we predict exact integers?

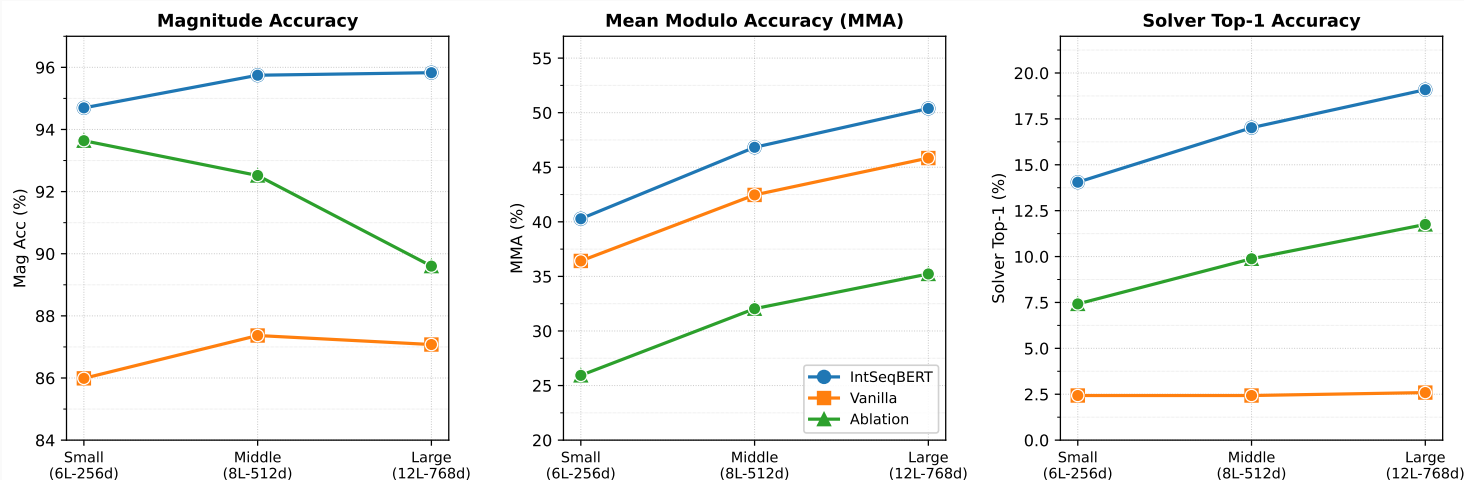
Q1: overall gains. Q2: gains by value bucket. Q3: exact prediction.

Q1 — Does It Help? Yes, Consistently

Model size	Model	Mag Acc	Sign Acc	MMA
Large	IntSeqBERT	95.85	98.54	50.38
Large	Vanilla	86.97	97.66	45.85
Large	Ablation (no modulo)	89.70	98.29	35.22

- **First, the dual stream improves both magnitude and residue prediction.** The Large model reaches **95.85%** magnitude accuracy (**+8.9 pt** over Vanilla) and **50.38%** MMA (**+4.5 pt**).
- **Removing the modulo stream makes both scores worse.** In the ablation, MMA falls by **15.2 pt** and magnitude accuracy by **6.2 pt**. Residues limit the possible integer values, helping both predictions.
- **The same pattern holds for the Small and Middle models.**

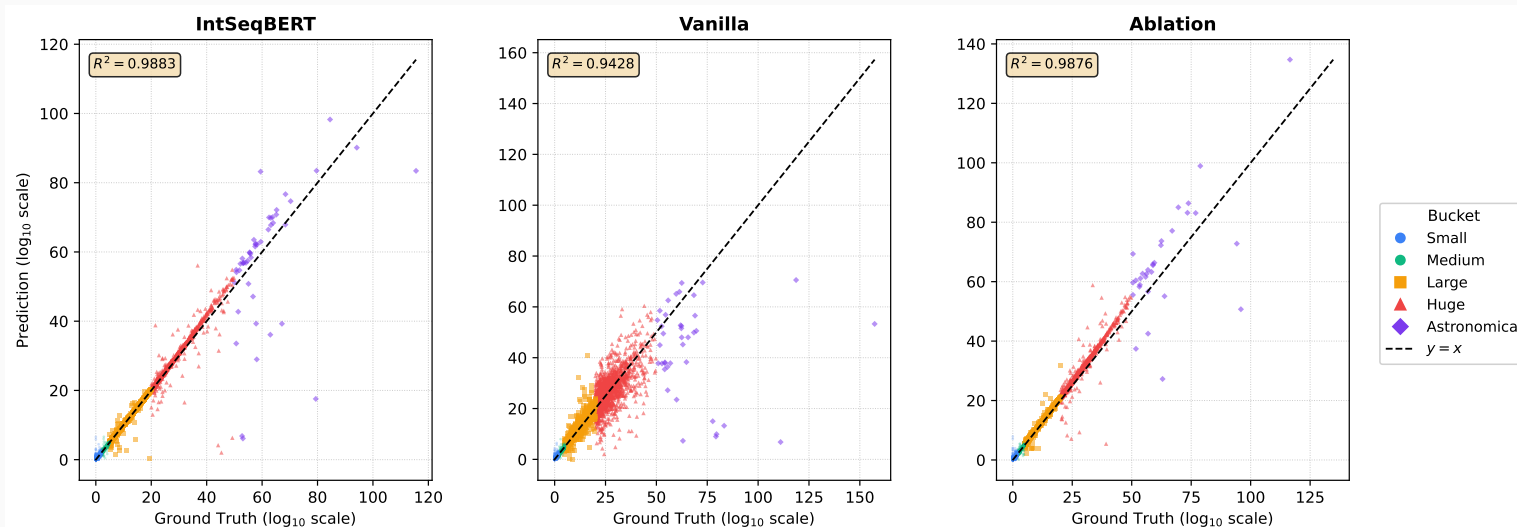
Full Results & Scaling Behaviour



Model size: Small → Middle → Large

- **IntSeqBERT has the highest modulo accuracy at every model size.** For MMA, the ranking is always IntSeqBERT > Vanilla > Ablation.
- **Larger models help residue prediction more than magnitude prediction.** From Small to Large, MMA gains **10.0 pt**; Mag Acc gains **1.1 pt**.
- Solver Top-1 also gains **5.0 pt**: 14.05 → 19.09.
- We see no clear sign of overfitting: Train 1.00 / Val 1.01 at epoch 200. We plan larger-scale retraining.

Q2 — Where? Exactly Where Tokenisation Breaks



Prediction error (MSE)

Value bucket	IntSeq	Vanilla	Abl.
Small	0.111	0.138	0.103
Large	0.162	2.100	0.381
Huge	2.082	22.73	5.021

- Next, the clearest gains are in the **Large** value bucket and **above**. Vanilla uses UNK beyond its vocabulary. In the Large bucket, its MSE is **13× ours**. Overall R^2 is **0.988** for IntSeqBERT and 0.943 for Vanilla.
- **Overall scores can hide these differences.** Many OEIS sequences grow slowly, so easier cases strongly affect those scores.

Q3 — The Strictest Test: Exact Next-Term Prediction

Model size	Model	Top-1	Top-10
Large	IntSeqBERT	19.09	26.23
Large	Vanilla	2.59	3.80
Large	Ablation	11.75	21.79

- **Finally, we test whether the full system predicts the exact next term.** We predict the last term of **10,000** sequences kept separate for testing.
- **IntSeqBERT reaches 19.09 percent, compared with 2.59 percent for Vanilla.** This is a **7.4-fold improvement**.
- Top-1 accuracy is **68.3%** in the Small value bucket and **20.8%** in Medium. Vanilla scores **0%** beyond its vocabulary.
- **However, our exact accuracy also falls to near zero for large values.** For the Large bucket and above, combining uncertain predictions from many moduli remains difficult at $\text{MMA} \approx 50\%$.

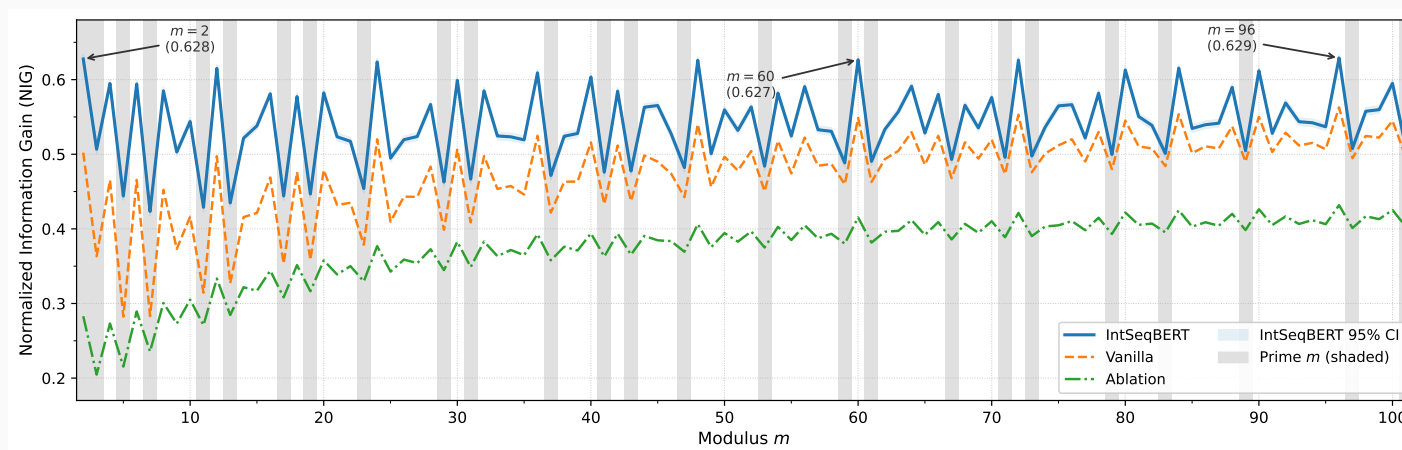
What Does 19% Mean? — Better, but Still Low

- So the improvement is large, but the absolute accuracy is still low. Top-1 rises from 2.59% to 19.09%, about 7.4 times the Vanilla score.
- Only about one in five predictions is exactly correct. In the Large value bucket and above, exact accuracy is still close to zero.

What this tells us

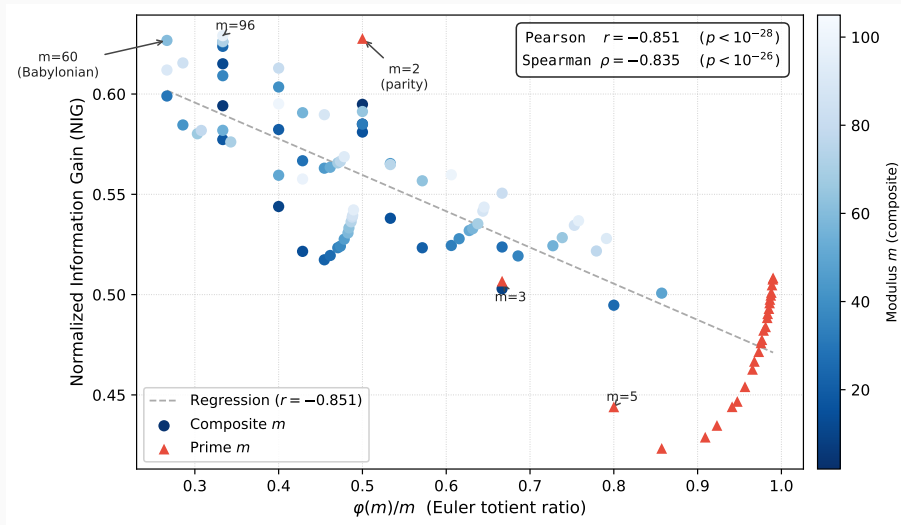
The modulo stream helps, but exact prediction of OEIS terms remains difficult.

But What Did the Network Learn? The NIG Spectrum



- We also ask what arithmetic structure the model has learned. NIG measures this for each modulus from 2 to 101, beyond overall accuracy.
- IntSeqBERT learns more at every modulus, with a clear pattern of peaks. It is above both baselines across the entire NIG spectrum.
- The highest peak is at modulus 96, followed by modulus 2. This holds across all models and sizes: $96 = 2^5 \times 3$ (NIG = 0.629). Mod 2 captures **parity**, which is common in the OEIS.
- Mod-2 accuracy is **85.7** for IntSeqBERT, 81.4 for Vanilla, and 72.1 for Ablation.

Finding — Higher NIG with a Lower Totient Ratio



Observation

We find higher NIG with a lower totient ratio.

Euler's ratio is $\varphi(m)/m = \prod_{p|m} (1 - 1/p)$.

Pearson $r = -0.851$ ($p < 10^{-28}$).

This is an experimental result, not a theorem.

Interpretation — CRT aggregation

One modulus can combine information from several smaller moduli.

For example, $x \bmod 96$ gives $x \bmod 2, 4, 8, 16, 32$ and $x \bmod 3$ at the same time.

Next steps

This suggests testing more composite moduli.

- Test highly composite numbers beyond 101 before adding more primes.
- Use these arithmetic patterns to compare sequences or datasets.

Positioning in Computer Mathematics

- This brings us back to our goal of finding mathematical connections.
- Program synthesis on the OEIS seeks a sequence's generation rule. Examples include **Alien Coding**, **QSynt**, and **Learning Conjecturing**.
- Our representation can support program synthesis by finding shared arithmetic structure. We learn this structure across the whole corpus.
 - Synthesis asks: *“What program generates this sequence?”*
 - We ask: *“Which sequences are number-theoretically similar, even when they are generated in different ways?”*
- Pretrained embeddings could **guide the search** in program synthesis and help us **find similar sequences**.
- These similarities could help us suggest new conjectures.
- Our code, preprocessing, and evaluation are available at github.com/aabaa/IntSeqBERT.

Conclusion

Four main points:

1. **First, the modulo spectrum makes arithmetic structure visible in the input.** A ring homomorphism preserves multiplication at depth zero.
2. **Second, it improves prediction at every model size.** Exact next-term accuracy improves by $7.4\times$ over Vanilla.
3. **Third, composite moduli combine useful arithmetic information.** NIG is negatively correlated with $\varphi(m)/m$ ($r = -0.851$).
4. **However, exact prediction remains difficult.** This supports further work on neural-symbolic methods.

Next, we want to improve decoding and use similar sequences to suggest conjectures.

We plan approximate CRT for large integers, highly composite moduli beyond 101, larger-scale retraining, and similarity retrieval & conjecture generation.

Thank you for listening. — github.com/aabaa/IntSeqBERT

Backup — Why Not Digit-Based Tokenisation?

- **Another option is to generate integers one decimal digit at a time.** Each digit is a token. We have not compared this approach experimentally in this paper.
- **Our input gives the model residues directly.** We target the **number-theoretic structure of the integer itself**. With digit tokens, the model must calculate residues from the digits.
- **Digit-by-digit generation also changes the prediction task.** It predicts a sequence of digits, while our model predicts one integer value.
- We plan to compare the two settings under the same conditions.

Backup — Limitations

- **Large integers remain difficult because residue predictions are uncertain.** In **13.4%** of Solver calls, no valid candidate is found (`none` mode). Approximate CRT or less strict residue constraints may help.
- **The dataset also affects our overall scores.** Many `nonn` sequences help sign accuracy; many slowly growing sequences help magnitude accuracy.
- **Our experiments use limited compute and one random seed.** We use one **8 GB GPU**, seed **42**, and a vocabulary of **20,003** tokens. Vanilla uses the *FACT architecture* with our settings; we do not reproduce FACT's reported results.

Backup — How Hard Is Multiplication for Neural Networks?

- **The answer depends on the circuit type and the training method.** In circuit theory, n -bit multiplication is not in AC^0 (constant-depth AND/OR). Polynomial-size threshold circuits can do it with **3 layers**, and 3 is optimal. Iterated products need only 4 layers (Siu & Roychowdhury, *SIAM J. Discrete Math.*, 1994).
- Log-precision **Transformers** are upper-bounded by uniform TC^0 (Merrill & Sabharwal 2023). This connects multiplication to model limits.
- **Experiments show that learning multiplication can still be difficult.** GPT-4 zero-shot accuracy falls from **59%** for 3×3 -digit products to **4%** for 4×4 -digit products (Dziri+, *Faith and Fate*, NeurIPS 2023).
- A **2-layer Transformer** on modular arithmetic generalises only after a very long training period: **grokking** (Power+ 2022, arXiv:2201.02177).
- Standard training fails at multi-digit multiplication even at **90B** parameters. A **12-layer model** succeeds on **15-digit products** only with implicit-CoT training. Its attention forms a **binary tree of partial products** (mechanistic analysis, arXiv:2510.00184, 2025).